

REPORT 1:

Educational Content Recommendation Sorting Using Merge Sort

Title:

Educational Content Recommendation Sorting Using Merge Sort

Objective

To analyze the application of Merge Sort in sorting educational content based on learner ratings and completion statistics for efficient recommendation delivery.

Problem Statement

Online learning platforms maintain large collections of courses. Before recommending content to learners, courses must be sorted according to ratings, completion rates, and popularity. An efficient sorting technique is required to handle large datasets while preserving the order of courses having identical scores.

Algorithm Used

Merge Sort

Merge Sort follows the Divide and Conquer paradigm:

1. Divide the dataset into smaller sublists.
2. Recursively sort each sublist.
3. Merge the sorted sublists into a final sorted list.

Implementation

The recommendation scores of courses are sorted using Merge Sort to generate ranked educational content recommendations.

Working Process

1. Collect course recommendation scores.
2. Divide the dataset into two halves.
3. Recursively divide until single elements remain.
4. Merge sublists in sorted order.
5. Generate the final recommendation list.

Issues Identified

Stable Ordering

Merge Sort is stable, meaning courses with identical recommendation scores retain their original order.

Duplicate Score Handling

Multiple courses may have the same rating. Merge Sort correctly places duplicate values while maintaining consistency.

Scalability

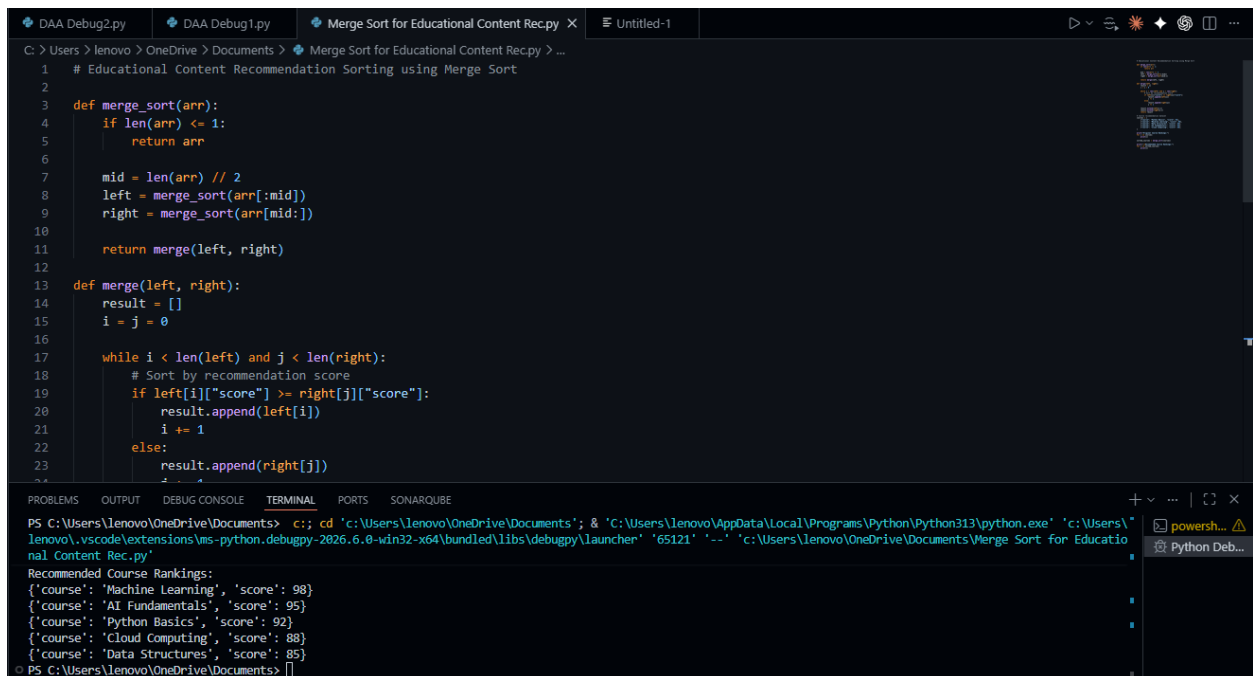
The algorithm performs efficiently even when thousands or millions of courses are processed.

Time Complexity Analysis

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Space Complexity

$O(n)$



```
C: > Users > lenovo > OneDrive > Documents > Merge Sort for Educational Content Rec.py > ...
1 # Educational Content Recommendation Sorting using Merge Sort
2
3 def merge_sort(arr):
4     if len(arr) <= 1:
5         return arr
6
7     mid = len(arr) // 2
8     left = merge_sort(arr[:mid])
9     right = merge_sort(arr[mid:])
10
11     return merge(left, right)
12
13 def merge(left, right):
14     result = []
15     i = j = 0
16
17     while i < len(left) and j < len(right):
18         # Sort by recommendation score
19         if left[i]["score"] >= right[j]["score"]:
20             result.append(left[i])
21             i += 1
22         else:
23             result.append(right[j])
24             j += 1
25
26     result.extend(left[i:])
27     result.extend(right[j:])
28
29     return result
30
31 # Initial list of course recommendations
32 initial_list = [
33     {'course': 'Machine Learning', 'score': 98},
34     {'course': 'AI Fundamentals', 'score': 95},
35     {'course': 'Python Basics', 'score': 92},
36     {'course': 'Cloud Computing', 'score': 88},
37     {'course': 'Data Structures', 'score': 85}
38 ]
39
40 # Sort the list using Merge Sort
41 sorted_list = merge_sort(initial_list)
42
43 # Print the sorted list
44 print("Sorted Course Recommendations:")
45 for course in sorted_list:
46     print(course)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE

```
PS C:\Users\lenovo\OneDrive\Documents> c:\; cd 'c:\Users\lenovo\OneDrive\Documents'; & 'C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '65121' '--' 'c:\Users\lenovo\OneDrive\Documents\Merge Sort for Educational Content Rec.py'
Recommended Course Rankings:
{'course': 'Machine Learning', 'score': 98}
{'course': 'AI Fundamentals', 'score': 95}
{'course': 'Python Basics', 'score': 92}
{'course': 'Cloud Computing', 'score': 88}
{'course': 'Data Structures', 'score': 85}
PS C:\Users\lenovo\OneDrive\Documents>
```

Advantages

- Stable sorting algorithm.
- Predictable performance.
- Handles duplicate values efficiently.
- Suitable for large datasets.
- Efficient for external storage applications.

Result

Educational content was successfully sorted according to recommendation scores. The sorted list improves recommendation quality and user experience.

Conclusion

Merge Sort is highly suitable for educational recommendation systems because it guarantees stable ordering, handles duplicate scores effectively, and maintains $O(n \log n)$ complexity regardless of input distribution.